



**Team
AXIOMATA**

SYSTEM ARCHITECTURE

Developed By
Team AXIOMATA

Developed for
Waypoint Group (Pvt) Ltd.

Access to RELAY

1. Design Philosophy - One Service, One Origin

Relay runs as a single container that serves the React web app (static files), the FastAPI REST/SSE API and the Python planning engine from one origin. This eliminates CORS complexity, allows session cookies to work without special configuration, and gives judges a single URL. The planner and ML predictor are imported in-process, there is no inter-service network hop.

PostgreSQL is the sole persistence layer: it holds all operational state, the event audit log, live-update notifications (LISTEN/NOTIFY), the per-workspace virtual clock and binary media (photos, signatures). One container can be killed at any moment, and another one continues seamlessly.

2.The system on one page

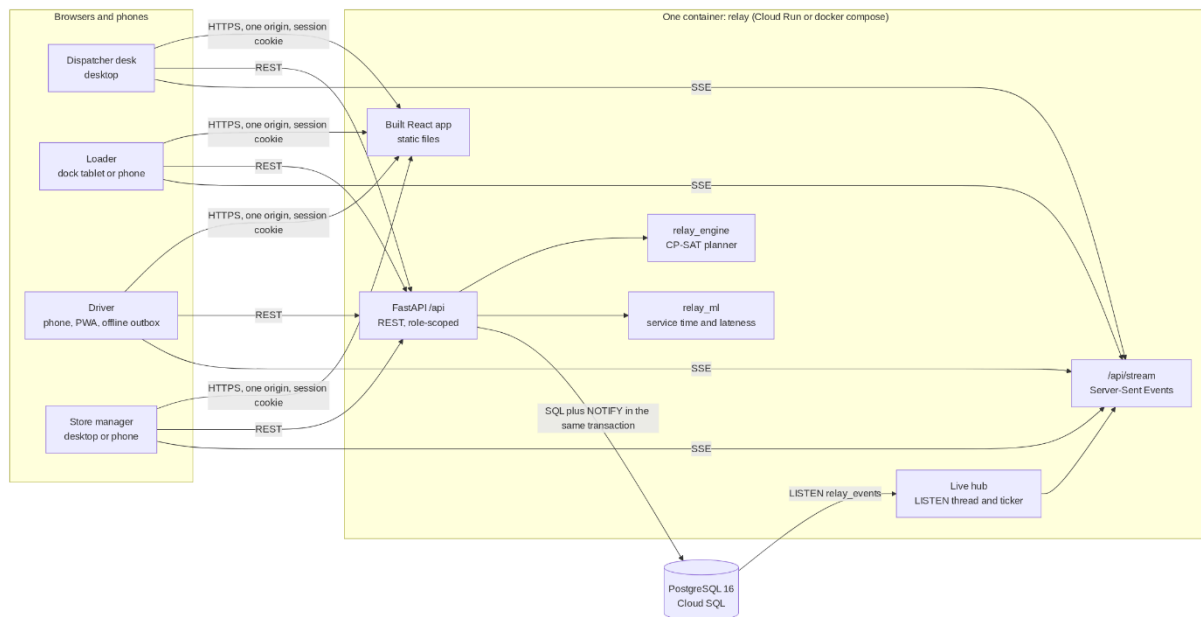


Figure 1: The system on one page

3. Component Map

Component	Location	Notes
React Web App	apps/web	React 19, TypeScript, Vite, PWA, each role is a lazy chunk with its own CSS
FastAPI API	apps/api/relay_api	REST + SSE endpoints, role-scoped, sync route handlers run in the thread pool
Planning Engine	packages/engine/relay_engine	Pure Python, OR-Tools CP-SAT
ML Predictor	packages/ml/relay_ml	LightGBM for service-time and lateness predictions; heuristic fallback
Live Hub	apps/api/relay_api/realtime.py	One LISTEN thread per process; fans out Postgres notifications to SSE streams
PostgreSQL 16	Cloud SQL / Docker	Schema via Alembic; 10 global + 16 workspace tables

4. Request Lifecycle

Every write request follows a consistent pipeline that guarantees atomicity between the business change and the live-update notification:

- 1. Browser sends POST with session cookie and X-Relay-Client: web header.
- 2. FastAPI verifies the signed JWT, loads user/workspace context, and checks CSRF.
- 3. A role dependency (dispatcher / loader / driver / store) enforces access.
- 4. The domain function runs business rules, raises a human-readable error on refusal.
- 5. record() appends an event row AND queues pg_notify('relay_events', ...) in the same transaction.
- 6. COMMIT - the notification is delivered only if the write succeeded.
- 7. The LISTEN thread forwards the notification to all matching SSE subscribers.
- 8. Affected browsers receive an 'invalidate' event and refetch their read models over REST.

5. Live Update Architecture

Live updates use PostgreSQL LISTEN/NOTIFY rather than a separate message broker. This means every Cloud Run instance hears every change without sticky sessions or Redis. Burst coalescing (250 ms window) prevents flooding when a plan publish touches many topics.

Topic	Meaning	Subscribers
plan	Plan or trip progress changed	All roles
clock	Virtual demo clock moved	All roles
depot:<id>	Dock queue / plan change at that depot	Loaders at that depot
vehicle:<id>	Van run update	That van's driver
outlet:<id>	Store orders / notices	That store's manager
dispatch (all)	Catch-all for dispatcher canvas	Dispatcher only

6. Authentication & Security

Sign-in issues an `httpOnly`, `SameSite=Lax` JWT cookie (scrypt password hashing). Every write endpoint requires the custom header `X-Relay-Client: web` to block cross-site form attacks. Domain functions enforce row-level ownership: a driver touches only their vehicle's trips, a loader only their depot's dock, a store only their own outlet's orders. The UI never controls access - the server does.

7. Workspaces & Sandboxes

Every operational row carries a `workspace_id`. The shared demo is 'main'; any visitor can create a private sandbox from the sign-in page (code like `RLY-7K2Q`). Sandboxes allow judges to explore the system without colliding. 'Reset demo' rebuilds a workspace from the seed in seconds. The Playwright e2e suite runs in its own sandbox so it is safe to point at production.

8. Deployment Pipeline

The Docker image is built in two stages: Node builds the React app; Python 3.12 slim runs it. On startup the container runs 'alembic upgrade head' then seeds missing data (both protected by Postgres advisory locks so multiple instances can start concurrently). Cloud Run hosts the service in `asia-south1` with a minimum of one instance.

Step	Tool
Source upload	make deploy (Cloud Build) or GitHub Actions (Workload Identity, no keys)
Image build	Root Dockerfile (two-stage: Node → Python slim)
Image registry	Artifact Registry
Runtime	Cloud Run, <code>asia-south1</code> , min 1 instance
Database	Cloud SQL PostgreSQL 16 via <code>/cloudsql</code> socket
Secrets	Secret Manager (<code>DATABASE_URL</code> , <code>SECRET_KEY</code>)
Health check	GET <code>/api/health</code> - reports DB, seed, predictor and revision

9. Quality Gates

Gate	What It Proves
Ruff · ESLint · Prettier · tsc --noEmit	Style and type correctness
Engine unit tests	Trip-time formula, every allocation rule, all three policies, explanations, capacity rounding
API tests (real Postgres)	Full walkthrough over HTTP, idempotent sync, security scopes, ordering cutoff
make check-plan	Demo day plan passes the organisers' validator under every policy
Playwright e2e	Four-role walkthrough in real browsers including dark-van and receipt scenarios
Docker Compose CI	Fresh clone starts with docker compose up and passes smoke tests